

Validator-Constrained Modular Interface Prediction for Embodied Agent Decision-Making

Xiang Li¹ Peng Wang¹ Lei Meng¹

¹School of Software, Shandong University

xiangli@mail.sdu.edu.cn, wpironman2025@gmail.com,
lmeng@sdu.edu.cn

Corresponding author: xiangli@mail.sdu.edu.cn

Abstract

We describe the CREAM submission to the Embodied Agent Interface Challenge, which evaluates goal interpretation, subgoal decomposition, action sequencing, and transition modeling in BEHAVIOR and VIRTUALHOME environments. Rather than using one general prompting strategy for all modules, our system treats each module as a typed structured-generation problem with its own prompt format, candidate generator, and validator. For BEHAVIOR, compact prompt serializations and AxisTilted2 Qwen3-0.6B specialists provide high-precision outputs, with prompt-derived direct-goal completion for subgoal decomposition and balanced-block cleanup for transition modeling. For VIRTUALHOME, we combine Qwen3-based LoRA generation, multi-candidate voting, structural candidate selection, and conservative simulator-aware repairs; transition modeling uses an inference-only Qwen3-0.6B official-domain LoRA and whitespace-only PDDL postprocessing. The best submitted run obtains an overall score of 70.695, with 81.825 on BEHAVIOR and 65.925 on VIRTUALHOME. Our results suggest that embodied interface prediction benefits less from broad free-form reasoning than from task-specialized generation followed by narrow, auditable validation.

1 Introduction

The Embodied Agent Interface (EAI) benchmark decomposes household embodied reasoning into four text-to-structure modules: Goal Interpretation (GI), Subgoal Decomposition (SD), Action Sequencing (AS), and Transition Modeling (TM) [4]. The same four interfaces are evaluated in two simulator families, BEHAVIOR and VIRTUALHOME. This design makes failures easier to localize than in end-to-end embodied control, but it also makes the output format unusually brittle: a response can express the intended behavior and still fail because an object identifier, predicate, JSON wrapper, action order, or PDDL parenthesis does not match the evaluator.

Our submission was developed around this interface brittleness. We found that large-model prompting alone was not the most reliable path: some modules need creative candidate generation, while others are almost deterministic once the action or predicate schema is known. We therefore use a portfolio of task-specialized pipelines. Small Qwen3-family models and LoRA adapters are used where the mapping is learnable from prompt patterns; Qwen3-32B candidates are used where broader commonsense helps; and validators constrain all final outputs to the official row order and schema.

The main design principle is conservative intervention. Generation is allowed to vary, but repair rules are only applied when they are tied to explicit interface constraints or repeated evaluator failures. This avoids replacing useful model-specific content with broad templates. In the released reproduction package, each module has a standalone entrypoint, the model weights are copied as real files rather than symlinks, and the Docker image version is fixed.

Our contributions are:

- a modular eight-output EAI system covering all BEHAVIOR and VIRTUALHOME GI, SD, AS, and TM tasks;
- compact BEHAVIOR prompt construction that extracts only dynamic fields from the official prompts while preserving the official `{identifier, llm_prompt}` row format;
- task-specific candidate aggregation and repair, including LoRA voting for VIRTUALHOME GI, restore-GRAB repair for VIRTUALHOME SD, score-based candidate fusion for VIRTUALHOME AS, and inference-only official-domain LoRA generation for VIRTUALHOME TM;
- a reproducible inference package containing prompts, scripts, model weights, Docker image metadata, adapters, and manifest commands.

2 Related Work

Embodied interface benchmarks. VIRTUALHOME represents household activities as executable programs in simulated home environments [5]. BEHAVIOR and BEHAVIOR-1K emphasize object-centric household activities with predicate-state goals [7, 3]. The EAI benchmark standardizes these settings into modular interface prediction tasks, allowing separate evaluation of goal interpretation, decomposition, action sequencing, and transition modeling [4].

LLMs for structured embodied reasoning. Recent language models can perform high-level planning, but simulator-facing interfaces require exact symbolic outputs. Our system follows the practical view that LLMs are best used as structured generators whose outputs are checked against task-specific constraints. We use Qwen3-family models [6], LoRA adapters [1], and vLLM inference [2], but the key improvement comes from combining generation with narrow validators rather than from model scale alone.

3 Design Principles

Our system was built around two observations about the challenge setting. First, the official interface is strict: a response can be semantically reasonable and still fail because a JSON wrapper is malformed, an object identifier is not in the prompt, a PDDL block is unbalanced, or the output rows are not in the official order. Second, the eight modules do not have the same uncertainty profile. Some outputs require semantic search over possible goals or intermediate steps; others are better viewed as schema completion. We therefore separate semantic uncertainty, where multiple model candidates are useful, from interface uncertainty, where deterministic validation and repair are more important.

Reproducible preprocessing. Prompt rewriting is a powerful intervention, but it can become a hidden source of performance if it is done manually. For BEHAVIOR we therefore make the compact prompt directory a generated artifact. The builder extracts dynamic fields from the official prompts, keeps the official filenames, and preserves the `{identifier, llm_prompt}` row schema. This gives the small AxisTilted2 specialists a shorter input while keeping the transformation auditable.

Narrow repair boundaries. We only apply repairs that can be explained by explicit interface constraints or repeated evaluator-visible failure modes. Broad replacement templates are avoided because they can erase useful model-specific detail. The BEHAVIOR D02 completion is derived from each compact prompt’s goal-state section and is applied only to short outputs with simple non-quantified direct goals; VIRTUALHOME SD restores only removed GRAB actions; and VIRTUALHOME AS uses a frozen v6.2 repair set with structural fallback. The design goal is not to make postprocessing invisible, but to make its scope small enough that every repair can be inspected.

Inspectability. Each module writes raw candidates, final outputs, and a report or manifest. Depending on the task, these reports record row counts, parse failures, identifier-order checks, self-loops, action-goal counts, PDDL balance, or candidate-selection decisions. This made debugging tractable:

Module	Main interface risk	Mitigation
GI	invalid objects, self-loops, excessive action goals	constrained parsing and candidate voting
SD	non-atomic subgoals, missing enabling actions	conservative normalization and narrow action restoration
AS	invalid ordering, missing setup actions, parser-empty outputs	structural scoring and frozen repairs
TM	malformed PDDL, wrong action order, near-miss predicates	balanced extraction or schema validation

Table 1: Interface risks that motivate the validator-first design.

Env.	Task	Generator	Aggregation / validation policy
BEHAVIOR	GI	AxisTilted2 Qwen3-0.6B specialist	compact prompts; JSON cleanup and schema validation
BEHAVIOR	SD	AxisTilted2 Qwen3-0.6B specialist	C02 compact output plus prompt-derived D02 direct-goal completion
BEHAVIOR	AS	AxisTilted2 Qwen3-0.6B specialist	compact output copied through the D02 BAS/BSD recipe
BEHAVIOR	TM	AxisTilted2 Qwen3-0.6B TM specialist	balanced PDDL block extraction and JSON wrapping
VIRTUALHOME	GI	Qwen3-0.6B LoRA	three-temperature voting with object/relation constraints
VIRTUALHOME	SD	Qwen3-32B-AWQ no-think	conservative normalizer plus restore-GRAB ablation
VIRTUALHOME	AS	AxisTilted2 32B LoRA + Qwen3-32B-AWQ	score-based fusion, frozen v6.2 repairs, and structural fallback
VIRTUALHOME	TM	Qwen3-0.6B official-domain LoRA	candidate validation, schema alignment report, whitespace-only PDDL postprocess

Table 2: Final module recipes. Each row is implemented as a standalone entrypoint in the reproduction package.

when a score changed, we could usually attribute it to generation, voting, repair, or final serialization rather than treating the pipeline as a black box.

4 Method

4.1 Overview

Each EAI module is treated as a mapping from a prompt to a structured object: JSON goals, a list of subgoals, an ordered action program, or a set of PDDL action schemas. The pipeline has three stages. First, a task-specific generator produces one or more raw candidates. Second, parsers convert raw text into typed intermediate representations and discard wrappers such as markdown fences or thinking traces. Third, validators enforce row ordering, object and predicate constraints, action ordering, JSON validity, and PDDL balance before the final file is written.

Table 2 summarizes the final recipe. Script names are included only to make the method traceable to the reproduction package; the method itself is defined by the generator, aggregation, and validation policy.

4.2 Validation Interface

The validators are task-specific but share a common contract. They must accept the official prompt file, preserve row order, produce one JSON output row per prompt row, and report failures rather than silently dropping examples. For GI, the validator decodes `llm_output` into node, edge, and action goals and checks object names, relation names, self-loops, and action-goal count. For SD, the validator reasons over predicate/action atoms and records invalid atoms, missing target states, and removed or restored actions. For AS, raw action strings are parsed into typed steps so that candidates can be scored by object validity, goal coverage, and length. For TM, the validator checks action names, balanced parentheses, required PDDL sections, and the absence of known invalid predicate variants.

This validation contract determines what the system is allowed to change. We allow repairs that restore a missing wrapper, normalize a predicate spelling, remove a self-loop, or reinsert an enabling action when the raw candidate contains evidence for it. We do not use old high-scoring submissions as fallback data. In the reproduction package, retained artifacts are limited to prompts, model weights, LoRA adapters, schema assets, Docker metadata, and scripts needed to rerun inference.

4.3 BEHAVIOR Modules

Compact prompt construction. For BEHAVIOR GI, SD, and AS we replace long official prompts with compact serializations that preserve only dynamic task fields. The construction script reads the official prompt JSON files and writes the same filenames under `llm_prompts_axis06b_behavior_compact/`. Output rows remain `[{identifier, llm_prompt}]`, so all downstream inference scripts can use the same interface. The three compact layouts are: GI serializes relevant objects, initial states, task name, and goal instructions; SD serializes task name, relevant objects, initial states, and goal states; AS serializes initial states, target states, and interactable objects. Fields are separated by a literal delimiter `-`. This reduces static instruction length and makes the small task-specialized models focus on scene state and target conditions. The compact representation is especially helpful for BEHAVIOR because the official prompts contain long static instructions, while the changing information is mostly the scene state and the target condition. Shortening the input reduces context pressure for 0.6B models and reduces the chance that the model copies instruction text into the answer. Since identifiers and filenames are unchanged, the same postprocessors can validate compact-prompt outputs against the official row ordering.

Goal interpretation. The BEHAVIOR GI pipeline uses the compact prompts with the `b_gi_Qwen-0.6B-FINALSUB` model. Decoding is deterministic. The postprocessor removes wrappers, parses JSON-like text, restores official prompt ordering, and validates the expected node/edge goal schema. Most remaining GI errors in development were interface errors rather than high-level commonsense errors: missing keys, invalid wrappers, or goals attached to objects that did not appear in the scene. Deterministic decoding is therefore a good fit, because sampling tends to introduce more formatting and object-name variance than useful semantic diversity for this small 100-row BEHAVIOR split.

Subgoal decomposition and action sequencing. For SD and AS we run the compact C02 configuration with `b_sd_Qwen-0.6B-FINALSUB` and `b_as_2_Qwen-0.6B-FINALSUB`. The SD output is then passed through a deliberately small D02 completion step. The completion script reads the compact SD prompt, extracts simple direct goal atoms from the goal-state section, and appends missing atoms only when both the prompt and the current model output are short. It skips quantified patterns such as `forall`, `exists`, `forpairs`, `forn`, and `fornucleated`, and it does not contain identifier-specific patches. AS uses the same C02 generation run and is copied into the final D02 candidate directory, keeping the BAS and BSD recipe coupled but auditable. This separation is important for attributing score changes. SD is sensitive to missing predicate atoms, while AS is sensitive to executable action order. The D02 completion changes only the SD file, so the BAS output remains a direct product of the compact model generation. In other words, the final BAS/BSR recipe is a shared generation pass followed by an SD-only, prompt-derived correction.

Transition modeling. BEHAVIOR TM remains model-generated. The TM specialist is run on official prompts, after which the postprocessor extracts balanced `(:action ...)` blocks, strips markdown or JSON wrappers, wraps the result as `{"output": ...}`, and checks that the generated action order matches the prompt. This keeps the model responsible for action-schema content while making formatting deterministic. The postprocessor deliberately does not replace the model’s action schemas with a global template. It only extracts balanced action blocks, removes wrappers, and checks action order. This keeps the model as the source of schema content while eliminating common serialization failures.

4.4 VIRTUALHOME Modules

Goal interpretation. VIRTUALHOME GI uses a Qwen3-0.6B LoRA adapter. We generate three candidates for each prompt using greedy, temperature 0.2, and temperature 0.6 decoding. The voting stage parses node, edge, and action goals separately, filters invalid objects and self-loops using prompt-derived constraints, caps action goals to avoid over-specification, and falls back only to a

prompt-derived task template when all candidates are empty. The script explicitly avoids fallback to previous submissions. The three candidates play different roles. Greedy decoding gives a stable base prediction, moderate sampling often recovers omitted state goals, and higher temperature sampling increases coverage but also increases hallucination risk. Voting after parsing lets the system use repeated evidence without accepting every sampled field. We vote node, edge, and action goals separately because action goals are sparse and have a different error profile from state goals.

Subgoal decomposition. VIRTUALHOME SD uses Qwen3-32B-AWQ in no-think mode. A conservative postprocessor normalizes JSON, predicate names, goal states, and action groups while recording invalid atoms and missing goals. We then build ablation candidates from the raw and normalized outputs and select the `baseline_restore_grab` variant, which restores only intermediate `GRAB ( . . . )` actions that were removed by normalization. The narrow restore policy preserves enabling actions without reintroducing a broad set of noisy extra actions. This rule reflects a common VIRTUALHOME pattern: many plans must first hold an object before placing, pouring, or interacting with it. A conservative normalizer that focuses on final goal states can remove such enabling actions. Restoring only `GRAB` actions is a compromise between semantic completeness and the risk of reintroducing all noisy actions from raw model outputs.

Action sequencing. VIRTUALHOME AS combines two complementary sources: an AxisTilted2 Qwen3-32B LoRA candidate and a Qwen3-32B-AWQ thinking-mode candidate. A parser maps action strings into typed action steps. The optimizer scores candidates using structural validity, object references, action-goal coverage, node/edge goal coverage, and length penalties. The selected v5 output is passed through a frozen v6.2 repair layer for recurring evaluator failures, including TV watching, drinking, putting food inside containers, switching lights, reading, phone interactions, and prefix goal trimming. The final script also applies a general structural fallback to the v5 candidate when the v6 repair degenerates into an empty output or a standalone `STANDUP` sequence; this fallback is structure-based rather than identifier-specific. The two candidate families fail differently. The LoRA candidate is usually concise and aligned with the expected action vocabulary, whereas the thinking-mode Qwen candidate sometimes includes useful setup actions but is more verbose. The selector therefore uses structural features rather than a fixed preference for either model. Freezing v6.2 was also a reproducibility choice: after selecting a repair family, the final result no longer depends on evolving exploratory ablation scripts.

Transition modeling. VIRTUALHOME TM uses an inference-only Qwen3-0.6B LoRA trained for official-domain action-schema completion. At reproduction time no training is run: the artifact set contains the base Qwen3-0.6B weights, the LoRA adapter, the schema library, the official-domain reference output used for reporting, and the VirtualHome PDDL domain file. For each action, the generator samples multiple schema candidates from action stubs and selects a validated candidate. The final postprocess is limited to whitespace normalization around PDDL expressions. The run manifest records row-level format checks, action-level alignment with the schema library, and the domain PDDL hash. In our reproduction check, the final VTM output matched the comparison VTM file on all 1500 rows after PDDL whitespace normalization, while 486 rows were byte-identical. This indicates that the remaining differences are formatting differences rather than changes in action-schema content.

5 Experiments

5.1 Setup

All outputs follow the official EAI JSON format and preserve prompt row order. BEHAVIOR tasks contain 100 evaluation rows per module; VIRTUALHOME tasks contain 1500 rows per module. The final reproduction package uses the Docker image `eai-vllm-qwen3:v0.11.1`. Large batch

Environment	Task	Official score	Primary metric	Secondary metric
BEHAVIOR	Goal Interpretation	99.3	Goal F1 99.3	–
BEHAVIOR	Subgoal Decomposition	86.0	Task SR 86.0	Exec SR 90.0
BEHAVIOR	Action Sequencing	92.0	Task SR 92.0	Exec SR 96.0
BEHAVIOR	Transition Modeling	50.0	F1 100.0	Planner SR 0.0
VIRTUALHOME	Goal Interpretation	83.1	Goal F1 83.1	–
VIRTUALHOME	Subgoal Decomposition	62.2	Task SR 62.2	Exec SR 81.1
VIRTUALHOME	Action Sequencing	68.4	Task SR 68.4	Exec SR 90.1
VIRTUALHOME	Transition Modeling	50.0	F1 100.0	Planner SR 0.0
BEHAVIOR average		81.825		
VIRTUALHOME average		65.925		
Overall		70.695		

Table 3: Official final submitted results. SR denotes success rate.

inference is sharded across four GPUs where needed, while the VIRTUALHOME TM reproduction uses a single GPU by default. We report the official scores from the final submitted run.

We evaluate the final files exactly as submitted, without adding hidden fallback outputs at packaging time. The final sample submission contains eight JSON files, one per module. In our local validation, each BEHAVIOR file has 100 rows and each VIRTUALHOME file has 1500 rows. This row-count check is simple but useful: many interface failures during development came from partial shard merges, accidental row reordering, or writing a report file where a submission file was expected.

5.2 Main Results

Table 3 gives the final official metrics. The BEHAVIOR modules are strongest overall, especially GI and AS. Both TM modules obtain perfect action-schema F1 but zero planner success, so the official TM score is 50.0 under the challenge metric. VIRTUALHOME remains harder, with action sequencing and subgoal decomposition benefiting most from postprocessing and repair.

5.3 Result Interpretation

The BEHAVIOR results show the advantage of task-specialized small models when the prompt distribution is compact and the output format is tightly controlled. GI reaches 99.3, and AS reaches 92.0 task success with 96.0 execution success. SD is lower at 86.0 task success, which is consistent with the fact that missing a small number of predicate atoms can change the task success metric even when the generated subgoal list is mostly correct.

VIRTUALHOME is harder because the held-out set is larger and because action programs interact more strongly with simulator affordances. GI benefits from voting and reaches 83.1 F1, while AS reaches 68.4 task success and 90.1 execution success. SD and AS have lower task success than their BEHAVIOR counterparts but relatively high execution success, suggesting that many outputs are executable but do not exactly satisfy all target conditions. This is precisely the regime where validator-guided repair is useful: it can recover missing setup actions or remove invalid structures, but it cannot fully solve semantic underspecification in the prompt.

Both TM modules expose a limitation of the official metric. The action-schema F1 is 100.0, but planner success is 0.0, giving an official TM score of 50.0. This means the surface-level schema fields match the expected form, but the schemas still do not lead to planner success under the downstream execution test. Future work should therefore include planner-in-the-loop validation for TM rather than relying only on action-block matching.

Component	Metric	Baseline → Ablated system	Baseline	With component	Δ
BEHAVIOR SD direct-goal completion	B-SD Task SR	571624 → 571413	72.0	86.0	+14.0
VIRTUALHOME TM official-domain inference	V-TM F1	571413 → 571837	50.7	100.0	+49.3
VIRTUALHOME GI LoRA voting	V-GI F1	571837 → 571955	53.7	83.3	+29.6
VIRTUALHOME AS frozen repair	V-AS Task SR	571955 → 571964	52.0	64.3	+12.3

Table 4: Single-component quantitative ablations from EvalAI submissions. Each row reports the directly affected module metric, not the full submission average.

5.4 Key Ablations

Table 4 reports single-component ablations derived from EvalAI checkpoints. Each row compares a baseline submission and a later submission that primarily changes one module, and reports the metric for that module rather than the full eight-task score. We include only the largest and clearest component-level changes: prompt-derived BEHAVIOR SD completion, official-domain VTM inference, VGI LoRA voting, and frozen VAS repair.

The largest single-module gain is VTM: switching to the official-domain inference recipe raises VIRTUALHOME TM F1 from 50.7 to 100.0. VGI voting is also large, improving VIRTUALHOME GI F1 by 29.6 points. The prompt-derived BEHAVIOR SD completion and the frozen VAS repair provide smaller but still substantial gains of 14.0 and 12.3 points on their respective task-success metrics.

5.5 Failure Modes

We categorize failures into four groups. The first group is serialization failure: malformed JSON, extra markdown, thinking traces, or incomplete PDDL blocks. These errors are largely addressed by no-think decoding and wrapper stripping, but they remain important because the evaluator cannot score an unparseable row. The second group is interface-valid but semantically weak output, such as a VIRTUALHOME plan that executes but omits an enabling action or a BEHAVIOR subgoal list that misses one final predicate. These errors require task-aware repair rather than generic formatting cleanup.

The third group is over-generation. GI candidates can add self-loop edges or too many action goals; AS candidates can include redundant action prefixes or unnecessary interactions. Voting and length-aware scoring are designed to control this failure mode. The fourth group is over-repair, where a postprocessor removes useful model detail or restores too many actions from a raw candidate. This is why our final repairs are narrower than several exploratory variants: the most stable gains came from bounded changes whose effect could be traced row by row.

5.6 Why Not a Single Unified Model?

A single unified model is attractive, but the final system suggests that the modules reward different inductive biases. BEHAVIOR compact prompts work well with small specialists because the evaluation split is small and the dynamic fields are highly regular. VIRTUALHOME GI benefits from LoRA adaptation and voting because goals are diverse but still structurally parseable. VIRTUALHOME AS benefits from combining a concise specialist with a broader reasoning model. TM benefits least from open-ended generation and most from schema-level validation. A unified model would still need these module-specific parsers and validators, so the engineering simplicity of one generator would not remove the need for typed output handling.

5.7 Ablation-Oriented Development

Although the final report focuses on the submitted configuration, the system was developed as a sequence of small ablations rather than as one monolithic script. For BEHAVIOR SD, the key comparison is between the C02 compact model output and the C02+D02 output. Because D02 is derived from the prompt goal-state section and is gated by direct-goal count and current-output

length, each changed row can be inspected as an added direct target atom rather than a rewritten decomposition. This style of bounded ablation was more useful than maintaining many large prompt variants, because it localized the effect of the change.

For VIRTUALHOME GI, ablations were organized around candidate sources and voting thresholds. Single-temperature candidates were useful diagnostics: greedy decoding exposed omissions, while higher-temperature decoding exposed hallucinated relations or excessive action goals. The final voting scheme is a compromise between those failure modes. Importantly, the voting script parses structured goals before aggregation; it does not vote over raw strings. This allows node, edge, and action goals to use different acceptance behavior while still producing a single JSON object in the official format.

For VIRTUALHOME SD and AS, we found that postprocessing ablations were best expressed as named candidate directories. The SD pipeline writes the raw Qwen32B output, a conservative normalized output, and restore variants. The final restore-GRAB candidate is therefore not a hidden edit but one point in a visible family of outputs. The AS pipeline similarly separates candidate generation, v5 score-based selection, v6 repair, and v6.2 frozen repair. This separation helped prevent exploratory repair code from silently changing the final submission after a configuration had been selected.

The common lesson is that ablations should follow the same interface discipline as final submissions. Each intermediate file should be a valid candidate submission with the same row count and identifiers, not an ad hoc debug format. That convention made it easy to compare variants, package the final run, and later extract a clean reproduction package that contains only the artifacts needed for inference.

6 Reproducibility

The public reproduction repository is organized around a relative-path artifact layout. It contains scripts, prompt directories, lightweight resources, artifact documentation, Docker image metadata, and a task manifest. Large model weights, LoRA adapters, Docker image archives, runtime outputs, and caches are handled as external artifacts and are documented separately from the source tree. Model directories are expected to contain real copied files or mounted artifact directories, not broken symlinks.

The compact BEHAVIOR prompt directory can be rebuilt with `scripts/make_axis06b_behavior_compact_prompts.py`. The VIRTUALHOME GI adapter is stored in the VGI artifact directory; the VIRTUALHOME TM adapter and schema assets are stored in the VTM artifact directory. The main entry commands are recorded in `MANIFEST.json`. Artifact paths can be overridden with environment variables, allowing the same scripts to run from local files, mounted storage, or downloaded model directories.

The reproduction package is intentionally an inference artifact, not a training archive. For example, the VTM reproduction uses the saved pre-trained adapter and does not rerun its training process. Similarly, VGI reproduction uses the saved LoRA adapter and regenerates candidates for voting. This keeps the artifact focused on reproducing the submitted outputs while still exposing all prompt preprocessing, inference, voting, and postprocessing code needed to audit the pipeline.

7 Limitations and Discussion

Our system is optimized for the official symbolic interface rather than for a general embodied controller. The repairs are intentionally narrow and may not transfer to unseen simulators without retuning. Several modules also rely on task-specific specialist models or adapters, so the pipeline is less elegant than a single end-to-end model. Finally, TM results show that schema F1 alone does not guarantee planner success; a future version should validate PDDL schemas by running them through the downstream planner during development.

Despite these limitations, the modular design made development and debugging tractable. Each module exposes raw candidates, normalized outputs, and reports, so score changes can be traced to

generation, voting, or repair. This is the main practical lesson from our submission: for embodied interface prediction, small task-specialized generators plus explicit validators can be more reliable than unrestricted large-model reasoning.

Another limitation is that our validation reports are stronger for format and interface checks than for semantic correctness. They can detect malformed objects, invalid action names, row mismatches, or PDDL balance errors, but they cannot always decide whether a plausible subgoal sequence is the best one for a household task. Closing this gap likely requires using the official evaluator more directly during candidate selection, especially for SD and AS where executability and task success can diverge.

8 Conclusion

We presented CREAM, a modular system for the EAI Challenge. The system combines compact prompts, Qwen3-family specialists, LoRA adapters, candidate voting, and conservative validator-guided repair. Its best final score of 70.695 is obtained with a fully reproducible inference package. The approach emphasizes auditable structure: generation proposes candidates, while validators and small repair layers enforce the simulator interface.

Acknowledgments

We thank the EAI Challenge organizers for providing the benchmark, evaluator, and report template. We also acknowledge the compute resources used for Qwen3/vLLM inference on L40S-class GPUs.

References

- [1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [3] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabriel Levine, Michael Lingelbach, Jiankai Sun, et al. BEHAVIOR-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation. In *Conference on Robot Learning*, pages 80–93, 2022.
- [4] Manling Li, Shiyu Zhao, Qineng Wang, Kangrui Wang, Yu Zhou, Sanjana Srivastava, Cem Gokmen, Tony Lee, Erran Li Li, Ruohan Zhang, et al. Embodied agent interface: Benchmarking llms for embodied decision making. *Advances in Neural Information Processing Systems*, 37:100428–100534, 2024.
- [5] Xavier Puig, Kevin Ra, Marko Boben, Jiaman Li, Tingwu Wang, Sanja Fidler, and Antonio Torralba. Virtualhome: Simulating household activities via programs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8494–8502, 2018.
- [6] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [7] Sanjana Srivastava, Chengshu Li, Michael Lingelbach, Roberto Martín-Martín, Fei Xia, Kent Vainio, Zheng Lian, Cem Gokmen, Shyamal Buch, C. Karen Liu, Silvio Savarese, Hyowon Gweon, Jiajun Wu, and Li Fei-Fei. BEHAVIOR: Benchmark for everyday household activities in virtual, interactive, and ecological environments. *arXiv preprint arXiv:2108.03332*, 2021.

A Team Members

Team name: CREAM.

Xiang Li is a Ph.D. student in the School of Software, Shandong University. His research interests include embodied agents, large language models, planning, and evaluator-guided model adaptation.

Peng Wang is a master’s student in the School of Software, Shandong University. His work focuses on reproducible experimentation, model inference pipelines, and embodied task planning.

Lei Meng is a professor in the School of Software, Shandong University. His research interests include software engineering, intelligent systems, and AI-driven automation.

B Entrypoints

The following commands reproduce the eight output files from the reproduction package:

- **B-GI/B-TM:** `scripts/run_axis06b_behavior_bgi_btm_repro.sh`.
- **B-AS/B-SD:** `scripts/run_axis06b_bas_bsd_c02_d02_repro.sh`.
- **V-GI:** `scripts/reproduce_vgi_qwen06b_sft97_vote.sh`.
- **V-SD:** `scripts/reproduce_vsd_baseline_restore_grab.sh`.
- **V-AS:** `scripts/run_vas_v6_repro.sh`.
- **V-TM:** `scripts/reproduce_qwen06b_vtm_official_domain_infer.sh`.